

Conversational AI for E-Commerce

A Vision-Based Approach via CLIP, FAISS, and Claude Sonnet

Generative AI: Principles and Applications Final

Ethan Chen, Gavin Kunish

May 2026

Abstract

The report outlines the design and implementation of a multimodal conversational chatbot for e-commerce product support. It combines CLIP embeddings, a FAISS vector index, and Claude claude-sonnet-4-6 to answer product related questions using both text and image inputs. Built on the Kaggle Amazon Product Dataset 2020 of 10,002 products. The full system is deployed as a Streamlit application that supports natural language queries, product image uploads, and three prompting modes (zero-shot, few-shot, and multi-shot).

Contents

1	Introduction	3
2	Related Work	3
2.1	CLIP	3
2.2	Retrieval-Augmented Generation	3
2.3	Large Language Models	3
3	Data	4
3.1	Dataset	4
3.2	Fields and Preprocessing	4
4	System Architecture	4
4.1	CLIP Model	4
4.2	FAISS Index	5
4.3	LLM and Prompting	5
5	Implementation	6
5.1	Index Building (<code>build_index.py</code>)	6
5.2	Retrieval Module (<code>rag.py</code>)	6
5.3	Streamlit Application (<code>app.py</code>)	6
6	Evaluation	6
6.1	Test Set Construction	6
6.2	Results	6
6.3	Discussion	7
6.4	Limitations of the Evaluation	7

7	Challenges and Solutions	7
8	Future Improvements	7
9	Conclusion	8

1 Introduction

Customer support in e-commerce is expensive and often bottlenecked by the sheer variety of product-related questions. A customer might ask about specifications, usage instructions, compatibility, or simply want to know what a product in an image is. Traditional text-only chatbots cannot handle the image input case, and retrieval-only systems cannot generate coherent, context-aware answers.

A multimodal Retrieval-Augmented Generation (RAG) system addresses both of these gaps. By using a vision-language model for retrieval and a large language model (LLM) for generation, the system can match a query whether it comes in as text or an image to relevant products in a catalog and then synthesize a helpful, grounded response.

The project builds such a system end-to-end:

- **Data:** Amazon Product Dataset 2020, 10,002 products
- **Encoder:** CLIP ViT-B/32 for shared text-image embeddings
- **Index:** FAISS IndexFlatIP for exact cosine-similarity search
- **LLM:** claude-sonnet-4-6 via the Anthropic API
- **Interface:** Streamlit web application

2 Related Work

2.1 CLIP

CLIP (Contrastive Language-Image Pre-training) [1] trains a text encoder and an image encoder jointly on 400 million image-text pairs from the web. The training objective pulls matching text-image pairs together in a shared embedding space while pushing non-matching pairs apart. The result is an encoder that can produce semantically comparable vectors for text and images—which is exactly what is needed for cross-modal retrieval.

2.2 Retrieval-Augmented Generation

RAG [2] combines a retrieval step with a generative model. At inference time, relevant documents are fetched from an external knowledge base and passed as context to the LLM. This keeps the LLM from hallucinating product facts it was never trained on, and makes the knowledge base easy to update without retraining.

2.3 Large Language Models

Modern LLMs like Claude can follow complex instructions, maintain multi-turn conversation history, and process image inputs natively. This makes them well-suited as the generation component in a multimodal RAG pipeline.

3 Data

3.1 Dataset

The Amazon Product Dataset 2020 is available on Kaggle. The version used here contains 10,002 products across several categories, with Toys & Games (66.6%) being the largest, followed by Home & Kitchen (7.1%) and Clothing, Shoes & Jewelry (6.3%).

3.2 Fields and Preprocessing

The dataset contains 28 columns. After reviewing them, the following five were chosen for building product descriptions:

Field	Usage
Product Name	Primary identifier; placed first in the description
Brand Name	Appended as “Brand: X”
Category	Top-level only (text before first pipe character)
Selling Price	Appended as “Price: X”
About Product	First 300 characters; pipe delimiters replaced with periods

Table 1: Product fields used for CLIP text descriptions

Fields like `Product Specification` and `Technical Details` were excluded because they contain heavy HTML-like markup that degrades CLIP text quality. The five selected fields cover the information customers most commonly ask about.

The combined description is passed through the CLIP processor with `truncation=True`, `max_length=77` to respect the model’s token limit. Image URLs are preserved in metadata but are not embedded at index time.

4 System Architecture

The pipeline has three stages: encoding, retrieval, and generation.

1. **Encoding.** At index build time, each product description is embedded with CLIP’s text encoder into a 512-dimensional L2-normalized vector. At query time, the user’s text or uploaded image is embedded with the corresponding CLIP encoder.
2. **Retrieval.** The query vector is compared against all 10,002 stored vectors using FAISS inner product search. The top-k most similar products are returned.
3. **Generation.** The retrieved product metadata (name, brand, price, description) is formatted into a context block and passed to Claude along with the user’s question. Claude generates a natural language response grounded in that context.

4.1 CLIP Model

`openai/clip-vit-base-patch32` was used. The ViT-B/32 variant processes images as 14×14 patches of 32×32 pixels and produces 512-dimensional embeddings for both modalities. Text and

image embeddings are projected into the same space via `text_projection` and `visual_projection` layers respectively.

One implementation challenge was that transformers version 5.5.0 changed the return type of `get_text_features()` and `get_image_features()`. These methods returned a `BaseModelOutputWithPooling` object instead of a tensor. The fix was to call the sub-models directly:

```
# Text embedding
out = model.text_model(input_ids=..., attention_mask=...)
feats = model.text_projection(out.pooler_output)
feats = feats / feats.norm(dim=-1, keepdim=True)

# Image embedding
out = model.vision_model(pixel_values=...)
feats = model.visual_projection(out.pooler_output)
feats = feats / feats.norm(dim=-1, keepdim=True)
```

4.2 FAISS Index

`faiss.IndexFlatIP` performs exact inner product search. Since all vectors are L2-normalized, inner product equals cosine similarity. The exact index is appropriate at 10k scale; for millions of products an approximate index (`IndexIVFFlat` or HNSW) would be needed.

4.3 LLM and Prompting

Claude claude-sonnet-4-6 was chosen because it supports both text and image in the same API request, handles multi-turn conversation history natively, and follows system prompts reliably.

Three prompting modes are implemented:

Mode	Description
Zero-shot	System prompt defines the role and task only. No examples.
Few-shot	System prompt includes 3 worked Q&A examples covering features, comparisons, and image identification.
Multi-shot	System prompt includes 5 examples covering price, age suitability, product comparison, usage, and color/variant queries.

Table 2: Prompt modes and their descriptions

The user question is always framed as: “Product context from our database: [context] Customer question: [question].” Conversation history is passed on every turn so follow-up questions work correctly.

5 Implementation

5.1 Index Building (`build_index.py`)

This one-time script loads the CSV, constructs product descriptions, encodes them in batches of 128, and saves the FAISS index and JSON metadata to disk. Total time on CPU is roughly 4–5 minutes for 10,002 products.

5.2 Retrieval Module (`rag.py`)

Module-level singletons hold the FAISS index, product metadata, CLIP model, and processor. They are loaded once on first call to `retrieve()` and reused for all subsequent calls. The `retrieve()` function accepts either a text query or a PIL image.

5.3 Streamlit Application (`app.py`)

The app has three tabs:

- **Chat:** Main chatbot interface. Users type questions or upload a product image. Retrieved products and their images are displayed below each assistant response.
- **Evaluation:** Shows the pre-computed Recall@1/5/10 metrics and a bar chart. A “Run Evaluation Now” button re-runs the evaluation live.
- **Documentation:** Architecture overview, data preprocessing details, model descriptions, and file structure.

`@st.cache_resource` is used for the RAG resources (loaded once per Streamlit server process). `@st.cache_data` is used for product image downloads.

6 Evaluation

6.1 Test Set Construction

200 products are sampled uniformly at random (seed=42) from the index. Each product’s exact name is used as the query. The ground truth is that the queried product should appear in the top-k retrieved results among the 10,001 competing distractors.

6.2 Results

Metric	Score
Accuracy@1	0.940
Recall@1	0.940
Recall@5	0.985
Recall@10	0.990

Table 3: Retrieval evaluation results (n=200 test queries)

A Recall@1 of 0.94 means that 94% of queries have the correct product as the single top result. Recall@10 of 0.99 means that 99% of queries have the correct product somewhere in the first 10

results. These are strong results for exact-name queries on a 10k-product index.

6.3 Discussion

The high recall is expected when the query is the product’s exact name, since CLIP encodes names consistently. The 6% miss rate at Recall@1 comes from a few failure modes: products with very generic names (e.g., “Blue T-Shirt”) that match multiple similar items, and products with names containing special characters or model numbers that CLIP tokenizes differently than expected.

6.4 Limitations of the Evaluation

This evaluation only covers text-to-text retrieval. Cross-modal retrieval (image query against text index) is harder to measure without a manually annotated image test set, which the Amazon Product Dataset does not include. Response quality (whether the LLM’s answer is accurate and helpful) is also not formally measured here and would require human evaluation or a dedicated benchmark.

7 Challenges and Solutions

Transformers API change. Transformers v5.5.0 changed the return signature of `get_text_features()` and `get_image_features()`, returning a dataclass instead of a tensor. The fix was to call `text_model` and `vision_model` directly and apply the projection layers manually. This is more explicit and version-stable.

Image URL availability. Amazon CDN image URLs from 2020 are frequently unavailable or return 403 errors. Rather than embedding images at index time (which would have failed for most products), image URLs are stored in metadata and fetched on-demand in the Streamlit app with error handling. This keeps the index fast to build while still displaying images when they are available.

CLIP token limit. CLIP’s text encoder has a hard limit of 77 tokens. Long product descriptions were silently truncated by the processor, sometimes cutting off key features. The fix was to pre-truncate the “About Product” field to 300 characters before concatenating, so the most important parts (name, brand, category, price) always fit within the token budget.

Streamlit session state with image uploads. Streamlit re-runs the entire script on each interaction. Without care, the uploaded file reference would reset between the upload and the chat submission. The fix was to read the file bytes immediately and store them in a local variable, and to call `seek(0)` after reading so the file object is still valid for display.

8 Future Improvements

Fine-tuned CLIP. The off-the-shelf CLIP model was not trained on e-commerce data. Fine-tuning on Amazon product image-description pairs (using the dataset’s image URLs) would improve cross-modal alignment for product-specific visual features like color, material, and form factor.

Image embedding at index time. Currently, only text embeddings are stored. Downloading and embedding product images would enable true image-to-image similarity search and improve image-query results.

Hybrid retrieval. Combining CLIP’s semantic search with keyword-based BM25 retrieval would handle queries like exact model numbers (“B07KMVJJK7”) that CLIP handles poorly because they are out-of-vocabulary.

Approximate indexing. At 10k products, brute-force `IndexFlatIP` is fast enough. For a production catalog with millions of products, an HNSW or IVF index would reduce query latency significantly.

Response evaluation. Adding an automated evaluation of LLM response quality using a judge model or RAGAS framework would give a more complete picture of end-to-end system performance beyond retrieval recall.

9 Conclusion

This project demonstrates that a relatively simple RAG pipeline CLIP embeddings, FAISS retrieval, and a capable LLM can power a practical multimodal e-commerce chatbot. The retrieval stage is highly accurate ($\text{Recall@10} = 0.99$) and the system handles both text and image queries cleanly. The main limitations are around image URL reliability and the lack of cross-modal evaluation data, both of which are dataset constraints rather than architectural ones.

The full source code, FAISS index, and metadata are included in the project submission. The Streamlit app can be run locally with `streamlit run app.py` after providing an Anthropic API key.

References

- [1] Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., ... & Sutskever, I. (2021). *Learning transferable visual models from natural language supervision*. Proceedings of the 38th International Conference on Machine Learning (ICML).
- [2] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). *Retrieval-augmented generation for knowledge-intensive NLP tasks*. Advances in Neural Information Processing Systems (NeurIPS).
- [3] Johnson, J., Douze, M., & Jegou, H. (2019). *Billion-scale similarity search with GPUs*. IEEE Transactions on Big Data.
- [4] Anthropic. (2024). *The Claude 3 Model Family: Opus, Sonnet, Haiku*. Technical Report.